- In your code
- In the Django Admin
- As a result of loading a fixture

All three of these cases need to be dealt with in your signal handler. Signal handlers take at least one argument: the sender of the signal. Specific signals may also send other parameters, so it is best practice to include `**kwargs` in your function signature. The `post_save` signal includes three additional arguments to the function:

- `created` is `True` if the save action created the object and `False` on object updates.
- `raw` is `True` if the save is from a fixture being loaded.
- `instance` contains the created object.

To write a signal that automatically creates a `UserProfile` object, you need a function registered as a signal handler for `post_save` against the `User` ORM object. If the function got called for (1) a newly created object that (2) wasn't created by loading a fixture, then you can create a corresponding `UserProfile` object.

There is one small thing that prevents it from being this simple. Remember the `StackedInline` form for the `UserProfile` object in the Django Admin? If you use this inline form, the Django Admin will create both the `User` and `UserProfile` objects at the same time, *before* the signal gets fired. When you created the `nicky` user account, if you did not associate her with any `Musician` or `Venue` objects, the `UserProfile` would not be created. If you had associated her, the `UserProfile` object would be created. To get around this, the signal handler checks for an already existing `UserProfile` object and only creates a new one if it doesn't already exist.

Listing 6.8 contains the code for automatically creating a `UserProfile` object when a `User` object first gets saved. Add this code to your RiffMates/bands/models.py file below the `UserProfile` object.

Listing 6.8 View with authorization code for musician profiles or bandmates

```
# RiffMates/bands/views.py
...
from django.db.models.signals import post_save
from django.dispatch import receiver
...
@receiver(post_save, sender=User)
def user_post_save(sender, **kwargs):
    # Create UserProfile object if User object is new
    # and not loaded from fixture
    if kwargs['created'] and not kwargs['raw']:
        user = kwargs['instance']
        try:
            # Double check UserProfile doesn't exist already
            # (admin might create it before the signal fires)
            UserProfile.objects.get(user=user)
```

Import the signal called when an ORM Model is saved.

Import the @receiver decorator.

Register this function to be called when a User object emits the post_save signal.

Only take action if the User object is new and not created by a fixture.

The newly created User object

Attempt to get an existing UserProfile object.